

SAIDONCLUB — ESPECIFICACIÓN TÉCNICA COMPLETA v2.0

Documento de Instrucciones para Agente OpenCode

Marketplace de Servicios Profesionales + Módulos Faltantes del Sistema

INSTRUCCIÓN PARA EL AGENTE: Este documento es tu guía de trabajo. Debes leer TODO antes de escribir una sola línea de código. Cada sección describe qué construir, por qué, cómo conectarlo con el sistema existente, y qué pruebas debes pasar. No saltes secciones. No asumas nada que no esté escrito aquí.

PARTE 0 — CONTEXTO Y ESTADO ACTUAL DEL SISTEMA

0.1 Stack Tecnológico Existente (NO CAMBIAR)

```
Framework:    Next.js 14 con App Router + Server Actions
Base de datos: PostgreSQL vía Prisma ORM
Auth:         Sistema existente con roles y PIN
Backend:      API Routes RESTful ya implementadas
MLM Engine:   @saidonclub/mlm-engine (paquete externo)
Monorepo:
  apps/web/           → Frontend Next.js
  packages/database/  → Prisma schema
  packages/mlm-engine/ → Motor MLM
  packages/rbac/       → Control de acceso
  packages/config-engine/ → Configuraciones
```

0.2 Lo que ya está implementado y NO debes tocar

- Sistema de autenticación con 9 roles (CLIENT, PREFERENTE, PIONERO, PROVIDER, ADMIN, SUPER_ADMIN, ACCOUNTANT, SUPPORT)
- Wallet con transferencias P2P, retiros, PIN de seguridad
- Motor MLM completo con genealogía, regalías, rangos, cierre semanal
- Marketplace de productos con carrito y 9 métodos de pago
- Dashboard de usuario con árbol de red visual

0.3 Lo que DEBES construir en este sprint

1. Marketplace de Servicios Profesionales — módulo completo (prioridad máxima)
2. Sistema KYC para Proveedores de Servicios — verificación profesional

- 3. **Sistema de Beneficiarios Familiares** — hasta 5 por cuenta
- 4. **Sistema de Código QR** — identificación en punto de atención
- 5. **Sistema de Agenda y Citas** — solicitud, confirmación, cancelación
- 6. **Formulario Bipartito Digital** — aceptación en el momento del servicio
- 7. **Sistema de Facturación con IVA** — para servicios prestados
- 8. **Sistema de Reseñas Bidireccional** — clientes ↔ proveedores
- 9. **Sistema de Moderación y Sanciones** — amonestaciones, suspensiones
- 10. **Documentos Legales Dinámicos** — términos, reglamentos, disclaimers
- 11. **Sistema de Notificaciones Multi-canal** — WhatsApp, email, Telegram, push
- 12. **Panel de Administración de Servicios** — aprobaciones, contabilidad interna

PARTE 1 — MODELO DE DATOS (PRISMA SCHEMA)

INSTRUCCIÓN: Agrega los siguientes modelos al archivo `packages/database/schema/prisma`. Usa la relación correcta con los modelos existentes (User, Wallet, etc.). Ejecuta `prisma migrate dev` después de cada bloque de modelos relacionados.

1.1 Perfil del Proveedor de Servicios

prisma

```

model ServiceProvider {
    id                String                @id @default(cuid())
    userId            String                @unique
    user              User                 @relation(fields: [userId], references:

    // === ESTADO Y VERIFICACIÓN ===
    status            ProviderStatus        @default(PENDING_KYC)
    // PENDING_KYC → PENDING_APPROVAL → ACTIVE → SUSPENDED_TEMP → SUSPENDED_PERM → REJ

    kycStatus         KycStatus             @default(NOT_STARTED)
    // NOT_STARTED → IN_PROGRESS → SUBMITTED → APPROVED → REJECTED → REQUIRES_UPDATE

    kycSubmittedAt    DateTime?
    kycApprovedAt     DateTime?
    kycApprovedByUserId String?
    kycRejectionReason String?

    // === INFORMACIÓN PROFESIONAL ===
    businessName      String                // Nombre del negocio o consultorio
    profession         String                // Médico, Abogado, Arquitecto, etc.
    professionCategory ProfessionCategory    // Ver enum abajo
    licenseNumber      String?              // Número de colegiatura o licencia
    licenseIssuedBy    String?              // Organismo que emitió la licencia
    licenseExpiresAt   DateTime?            // Fecha de vencimiento (si aplica)
    bio                String?              // Descripción profesional (max 1000 ch

    // === CONTACTO Y UBICACIÓN ===
    phone              String
    email              String
    whatsapp           String?
    telegram           String?
    website            String?
    instagram          String?
    facebook           String?
    tiktok             String?
    address            String?
    city               String?
    province           String?
    latitude            Float?              // Para búsqueda por mapa
    longitude           Float?

    // === FOTOS (URLs almacenadas en Supabase Storage) ===
    profilePhotoUrl    String?              // 1 foto de perfil principal
    personalPhotos     String[]             @default([]) // Hasta 3 fotos personales
    workPhotos         String[]             @default([]) // Hasta 3 fotos de trabajo
    businessPhotos     String[]             @default([]) // Hasta 3 fotos del local

```

```

    adPhotos                String[]                @default([]) // Fotos publicitarias ili

    // === DOCUMENTOS KYC (URLs) ===
    idDocumentUrl           String?                  // Cédula o pasaporte
    idDocumentBackUrl       String?                  // Reverso del documento
    professionalTitleUrl    String[]                  @default([]) // Títulos académicos
    certificationUrls       String[]                  @default([]) // Certificaciones adicionales
    selfieWithIdUrl         String?                  // Selfie sosteniendo cédula

    // === ACUERDO DE CONVENIO ===
    agreementNumber         String?                  @unique // "Convenio Nro 362535"
    agreementSignedAt       DateTime?
    agreementDocUrl         String?

    // === RELACIONES ===
    services                 ServiceListing[]
    appointments             Appointment[]
    providerReviews          ProviderReview[]         // Reseñas que recibió
    clientReviews            ClientReview[]           // Reseñas que dio a clientes
    warnings                 ProviderWarning[]

    createdAt               DateTime                  @default(now())
    updatedAt               DateTime                  @updatedAt

    @@map("service_providers")
}

enum ProviderStatus {
    PENDING_KYC
    PENDING_APPROVAL
    ACTIVE
    SUSPENDED_TEMP
    SUSPENDED_PERM
    REJECTED
}

enum KycStatus {
    NOT_STARTED
    IN_PROGRESS
    SUBMITTED
    APPROVED
    REJECTED
    REQUIRES_UPDATE
}

enum ProfessionCategory {
    HEALTH // Salud: médicos, odontólogos, psicólogos, fisioterapeutas, etc.

```

```
LEGAL          // Legal: abogados, notarios, asesores jurídicos
ARCHITECTURE    // Arquitectura e ingeniería civil
ENGINEERING     // Otras ingenierías
FINANCIAL       // Contadores, asesores financieros
EDUCATION       // Profesores, tutores, coaches
BEAUTY          // Estética, peluquería, spa
TECHNOLOGY      // Técnicos, programadores, diseñadores
HOME_SERVICES   // Plomería, electricidad, carpintería
OTHER           // Otras categorías
}
```

1.2 Listado de Servicios del Proveedor

prisma

```

model ServiceListing {
  id                String                @id @default(cuid())
  providerId        String
  provider           ServiceProvider       @relation(fields: [providerId], referen

  // === IDENTIFICACIÓN ===
  name              String                // Nombre del servicio. Ej: "Consulta M
  description        String                // Descripción detallada
  category           ServiceCategory      // Ver enum
  isActive           Boolean               @default(true)
  requiresApproval   Boolean              @default(false) // Si necesita revisión

  // === TRES NIVELES DE PRECIO (CRÍTICO) ===
  // Precio 1: Lo que ve el público en general
  publicPrice        Decimal              @db.Decimal(10, 2)

  // Precio 2: Lo que ve el cliente con membresía SaidonClub (visible en marketplace)
  memberPrice         Decimal              @db.Decimal(10, 2)

  // Precio 3: Precio interno del convenio (SOLO visible para proveedor y empresa)
  // Este precio es el que se usa para calcular la comisión de la empresa
  internalPrice       Decimal              @db.Decimal(10, 2)

  // === ESTRUCTURA DE COMISIÓN ===
  // La empresa cobra la diferencia entre internalPrice y lo que paga al proveedor
  // Ejemplo: memberPrice=$40, internalPrice=$30, empresa retiene $10 + IVA
  companyCommission   Decimal              @db.Decimal(10, 2) // Calculado: member
  commissionPercentage Decimal              @db.Decimal(5, 2) // Para reportes

  // === IVA ===
  ivaPercentage        Decimal              @db.Decimal(5, 2) @default(15.00) // I
  ivaIncluded           Boolean              @default(false) // Si el precio ya incl

  // === MODALIDAD DEL SERVICIO ===
  modality             ServiceModality      // PRESENCIAL, VIRTUAL, DOMICILIO
  duration              Int                  // Duración en minutos
  allowEmergency         Boolean              @default(false) // Si acepta citas de e
  emergencySurcharge     Decimal?            @db.Decimal(10, 2) // Recargo adicional

  // === PREPAGO / POSTPAGO ===
  requiresPrePayment    Boolean              @default(false)
  // true = se paga en plataforma ANTES del servicio (no cancelaciones ni devolucion
  // false = se paga directamente al proveedor DESPUÉS del servicio (con QR de confi

  // === RELACIONES ===
  appointments          Appointment[]

```

```

reviews                ProviderReview[]

// === APROBACIÓN DE CAMBIOS ===
pendingUpdate           Json?                // Cambios solicitados que esperan apro
updatedAt               DateTime              @updatedAt
createdAt               DateTime              @default(now())

@@map("service_listings")
}

enum ServiceCategory {
    MEDICAL_CONSULTATION
    DENTAL
    PSYCHOLOGY
    PHYSIOTHERAPY
    NUTRITION
    OPTOMETRY
    LEGAL_CONSULTATION
    NOTARY
    ARCHITECTURAL_DESIGN
    ENGINEERING_CONSULTING
    ACCOUNTING
    FINANCIAL_ADVISORY
    TUTORING
    COACHING
    HAIRCUT_STYLING
    AESTHETIC_TREATMENT
    MASSAGE_THERAPY
    WEB_DEVELOPMENT
    GRAPHIC_DESIGN
    TECHNICAL_SUPPORT
    PLUMBING
    ELECTRICAL
    CARPENTRY
    CLEANING_SERVICE
    OTHER
}

enum ServiceModality {
    PRESENCIAL           // En el local del proveedor
    VIRTUAL               // Videollamada
    DOMICILIO            // El proveedor va donde el cliente
}

```

1.3 Beneficiarios Familiares del Miembro

prisma


```

model FamilyBeneficiary {
    id                String                @id @default(cuid())
    memberId          String                // Usuario con membresía
    member            User                  @relation("MemberBeneficiaries", fields: ["memberId", "id"])

    // === DATOS PERSONALES ===
    firstName         String
    lastName          String
    relationship       FamilyRelationship  // SPOUSE, CHILD, PARENT, SIBLING, OTHER
    dateOfBirth       DateTime
    idDocumentNumber  String                // Número de cédula o pasaporte
    idDocumentType    IdDocumentType       @default(CEDULA)

    // === DOCUMENTOS (URLs) ===
    photoUrl          String?              // Foto del beneficiario
    idDocumentUrl     String?              // Imagen de la cédula
    idDocumentBackUrl String?              // Reverso

    // === ESTADO ===
    isActive          Boolean              @default(true)
    verifiedAt        DateTime?
    verifiedByUserId  String?

    // === QR INDIVIDUAL ===
    // El QR del titular cubre a todos sus beneficiarios
    // El proveedor ve la lista de beneficiarios al escanear el QR del titular

    appointments      Appointment[]        // Citas agendadas para este beneficiario

    createdAt         DateTime              @default(now())
    updatedAt         DateTime              @updatedAt

    @@map("family_beneficiaries")
}

enum FamilyRelationship {
    SPOUSE
    CHILD
    PARENT
    SIBLING
    OTHER
}

enum IdDocumentType {
    CEDULA
    PASSPORT
}

```

FOREIGN_ID

}

1.4 Sistema de Citas / Agenda

prisma


```

providerNotes                String?                // Notas internas del proveedor (pr

// === PRECIOS APLICADOS (se guardan en el momento de confirmar, no cambian) ===
appliedPublicPrice           Decimal                @db.Decimal(10, 2)
appliedMemberPrice           Decimal                @db.Decimal(10, 2)
appliedInternalPrice         Decimal                @db.Decimal(10, 2)
appliedIvaPercentage         Decimal                @db.Decimal(5, 2)
ivaAmount                    Decimal?               @db.Decimal(10, 2) // Calculado al
totalCharged                  Decimal?               @db.Decimal(10, 2) // Monto total
companyCommissionAmount      Decimal?               @db.Decimal(10, 2) // Comisión que
providerNetAmount            Decimal?               @db.Decimal(10, 2) // Lo que recib

// === PAGO ===
paymentMethod                 String?                // Si fue prepago en plataforma
paymentId                     String?                // ID de la transacción de pago
paymentStatus                 PaymentStatus?       @default(PENDING)
paidAt                        DateTime?

// === FORMULARIO BIPARTITO (se llena al momento del servicio) ===
bipartiteForm                  BipartiteForm?

// === FACTURA ===
invoice                       ServiceInvoice?

// === RELACIONES CON RESEÑAS ===
providerReview                ProviderReview?    // Reseña del cliente sobre el prov
clientReview                  ClientReview?        // Reseña del proveedor sobre el cl

createdAt                     DateTime                @default(now())
updatedAt                     DateTime                @updatedAt

@@map("appointments")
}

enum AppointmentStatus {
    PENDING_PROVIDER
    PROVIDER_RESPONDED
    PENDING_CLIENT
    CONFIRMED
    PAID
    IN_PROGRESS
    COMPLETED
    CANCELLED_CLIENT
    CANCELLED_PROVIDER
    NO_SHOW
    EMERGENCY
    DISPUTE

```

```
}

enum PaymentStatus {
    PENDING
    PROCESSING
    COMPLETED
    FAILED
    REFUNDED
}
```

1.5 Formulario Bipartito Digital

prisma

```

model BipartiteForm {
    id                                String                                @id @default(cuid())
    appointmentId                    String                                @unique
    appointment                       Appointment                       @relation(fields: [appointmentId],

    // === DATOS DEL SERVICIO PRESTADO (llena el proveedor) ===
    serviceDescription                String                                // Descripción detallada del servicio
    additionalServicesGiven           Json?                            // [{name, price}] si hubo extras no
    totalServiceValue                 Decimal                            @db.Decimal(10, 2)
    ivaApplied                        Decimal                            @db.Decimal(10, 2)
    totalWithIva                      Decimal                            @db.Decimal(10, 2)
    paymentTypeUsed                   FormPaymentType
    // PLATFORM_PREPAID = ya está pagado en plataforma
    // DIRECT_CASH = pagó en efectivo al proveedor
    // DIRECT_CARD = pagó con tarjeta al proveedor
    // PLATFORM_POINTS = pagó con puntos SaidonClub

    providerObservations              String?                            // Notas clínicas u observaciones (
    isObservationPrivate              Boolean                            @default(false)

    // === DECLARACIÓN DEL PROVEEDOR ===
    providerDeclares                  String                                // "Yo, [Nombre], con convenio Nro
    providerSignedAt                  DateTime?
    providerSignatureData              String?                            // Base64 de firma digital o "ACCEP
    providerIpAddress                  String?

    // === ACEPTACIÓN DEL CLIENTE ===
    clientDeclares                    String?                            // El cliente confirma recibir el s
    clientAcceptedAt                  DateTime?
    clientSignatureData                String?                            // Base64 de firma digital o "ACCEP
    clientIpAddress                    String?

    // === ESTADO DEL FORMULARIO ===
    formStatus                        BipartiteFormStatus @default(PROVIDER_FILLING)
    // PROVIDER_FILLING → PENDING_CLIENT_ACCEPTANCE → BOTH_SIGNED → DISPUTED

    // Si el cliente rechaza (no acepta), el proveedor cobra precio público
    clientRejectedAt                  DateTime?
    clientRejectionReason              String?

    createdAt                         DateTime                            @default(now())
    updatedAt                         DateTime                            @updatedAt

    @@map("bipartite_forms")
}

```

```
enum FormPaymentType {  
    PLATFORM_PREPAID  
    DIRECT_CASH  
    DIRECT_CARD  
    PLATFORM_POINTS  
}  
  
enum BipartiteFormStatus {  
    PROVIDER_FILLING  
    PENDING_CLIENT_ACCEPTANCE  
    BOTH_SIGNED  
    DISPUTED  
}
```

1.6 Facturación de Servicios

prisma

```

model ServiceInvoice {
  id                      String                      @id @default(cuid())
  appointmentId          String                      @unique
  appointment             Appointment                @relation(fields: [appointmentId],

  // === DATOS DE LA FACTURA ===
  invoiceNumber          String                      @unique // Número secuencial: SC-SF
  issuedAt               DateTime                    @default(now())

  // === PARTES ===
  providerLegalName      String                      // Nombre legal del proveedor
  providerRuc            String?                     // RUC del proveedor para facturaci
  clientLegalName        String                      // Nombre legal del cliente
  clientId_document      String                      // Cédula del cliente

  // === DETALLE FINANCIERO ===
  subtotal               Decimal                     @db.Decimal(10, 2) // Antes de IV
  ivaPercentage          Decimal                     @db.Decimal(5, 2) // 15% Ecuador
  ivaAmount              Decimal                     @db.Decimal(10, 2)
  total                  Decimal                     @db.Decimal(10, 2)
  companyCommission      Decimal                     @db.Decimal(10, 2) // Comisión re
  providerNetPayment     Decimal                     @db.Decimal(10, 2) // Lo que se l

  // === ESTADO ===
  invoiceStatus          InvoiceStatus               @default(ISSUED)
  pdfUrl                 String?                     // URL del PDF generado
  sentToClientAt         DateTime?
  sentToProviderAt       DateTime?

  @@map("service_invoices")
}

enum InvoiceStatus {
  ISSUED
  SENT
  PAID
  DISPUTED
  CANCELLED
}

```

1.7 Sistema de Reseñas Bidireccional

prisma


```

// Reseña del CLIENTE sobre el PROVEEDOR (visible para todos)
model ProviderReview {
    id                String                @id @default(cuid())
    appointmentId     String                @unique
    appointment       Appointment           @relation(fields: [appointmentId],

    clientId          String
    client            User                  @relation(fields: [clientId], refer

    providerId        String
    provider          ServiceProvider       @relation(fields: [providerId], ref

    serviceId         String
    service           ServiceListing       @relation(fields: [serviceId], refe

    // === CALIFICACIÓN ===
    rating            Int                    // 1 a 5 estrellas
    title             String?               // Título corto
    comment           String                // Comentario (mín 20 chars)
    isAnonymous       Boolean               @default(false)

    // === MODERACIÓN ===
    isVisible         Boolean               @default(true)
    moderationFlag    String?              // Si fue marcada para revisión
    moderatedByUserId String?
    moderatedAt       DateTime?

    createdAt         DateTime              @default(now())

    @@map("provider_reviews")
}

// Reseña del PROVEEDOR sobre el CLIENTE (visible para otros proveedores, NO para el
model ClientReview {
    id                String                @id @default(cuid())
    appointmentId     String                @unique
    appointment       Appointment           @relation(fields: [appointmentId],

    providerId        String
    provider          ServiceProvider       @relation(fields: [providerId], ref

    clientId          String
    client            User                  @relation(fields: [clientId], refer

    // === CALIFICACIÓN ===
    rating            Int                    // 1 a 5 estrellas

```

```
behaviorRating      Int           // Comportamiento del cliente
punctualityRating   Int           // Puntualidad
comment             String        // Solo lo ven otros proveedores
isRecommended       Boolean      @default(true)

// === MODERACIÓN ===
isVisible           Boolean      @default(true)
moderatedByUserId   String?

createdAt           DateTime     @default(now())

@@map("client_reviews")
}
```

1.8 Sistema de Sanciones y Amonestaciones

prisma

```

model ProviderWarning {
    id                String                @id @default(cuid())
    providerId        String
    provider           ServiceProvider      @relation(fields: [providerId], ref

    type              WarningSeverity
    // INFO → WARNING → SUSPENSION_TEMP → SUSPENSION_PERM

    reason            String
    details            String?
    issuedByUserId     String              // Admin que emitió
    issuedAt           DateTime            @default(now())
    expiresAt          DateTime?          // Para suspensiones temporales
    resolvedAt         DateTime?
    resolutionNotes    String?

    @@map("provider_warnings")
}

model ClientWarning {
    id                String                @id @default(cuid())
    clientId           String
    client             User                @relation(fields: [clientId], refer

    type              WarningSeverity
    reason            String
    details            String?
    issuedByUserId     String
    issuedAt           DateTime            @default(now())
    expiresAt          DateTime?
    resolvedAt         DateTime?
    resolutionNotes    String?

    @@map("client_warnings")
}

enum WarningSeverity {
    INFO              // Llamada de atención informativa
    WARNING            // Amonestación formal
    SUSPENSION_TEMP   // Suspensión temporal (días definidos)
    SUSPENSION_PERM   // Suspensión permanente
}

```

1.9 Disponibilidad del Proveedor (Calendario)

```
prisma

model ProviderSchedule {
  id          String          @id @default(cuid())
  providerId  String
  provider    ServiceProvider @relation(fields: [providerId], ref
  dayOfWeek   Int             // 0=Domingo, 1=Lunes, ..., 6=Sábado
  startTime   String          // "08:00"
  endTime     String          // "17:00"
  slotDurationMinutes Int     @default(60)
  isActive    Boolean         @default(true)
  breakStart  String?         // "12:00"
  breakEnd    String?         // "13:00"
  @@unique([providerId, dayOfWeek])
  @@map("provider_schedules")
}

model ProviderBlockedDate {
  id          String          @id @default(cuid())
  providerId  String
  provider    ServiceProvider @relation(fields: [providerId], ref
  date        DateTime
  reason      String?         // Vacaciones, feriado, etc.
  @@map("provider_blocked_dates")
}
```

1.10 Notificaciones del Sistema

```
prisma
```

```
model SystemNotification {
  id                      String                @id @default(cuid())
  userId                  String
  user                    User                  @relation(fields: [userId], referen

  type                    NotificationType
  title                   String
  body                    String
  data                    Json?                // Datos extra (appointmentId, etc.)
  channels                 String[]             // ["EMAIL", "WHATSAPP", "PUSH", "T

  // === ESTADO DE ENVÍO ===
  emailSentAt             DateTime?
  whatsappSentAt          DateTime?
  telegramSentAt          DateTime?
  pushSentAt              DateTime?
  readAt                  DateTime?

  createdAt               DateTime             @default(now())

  @@map("system_notifications")
}
```

```
enum NotificationType {
  APPOINTMENT_REQUESTED
  APPOINTMENT_RESPONDED
  APPOINTMENT_CONFIRMED
  APPOINTMENT_PAID
  APPOINTMENT_REMINDER_24H
  APPOINTMENT_REMINDER_1H
  APPOINTMENT_QR_SCANNED
  BIPARTITE_PENDING
  BIPARTITE_COMPLETED
  INVOICE_ISSUED
  REVIEW_REQUESTED
  REVIEW_RECEIVED
  WARNING_ISSUED
  SUSPENSION_NOTICE
  KYC_SUBMITTED
  KYC_APPROVED
  KYC_REJECTED
  NEW_SERVICE_PENDING_APPROVAL
  EMERGENCY_REQUEST
}
```

PARTE 2 — API ROUTES (BACKEND)

INSTRUCCIÓN: Crea todos los archivos de API dentro de `apps/web/app/api/`. Usa Server Actions donde sea apropiado. Cada endpoint debe validar: (1) autenticación, (2) rol autorizado, (3) datos de entrada, (4) reglas de negocio. Retorna siempre `{success, data, error}`.

2.1 API de Proveedor de Servicios

POST `/api/services/provider/register`

- **Quién puede llamarla:** Usuario con rol PROVIDER o quien solicite convertirse en proveedor
- **Qué hace:** Crea el registro inicial de ServiceProvider con status PENDING_KYC
- **Validaciones:** El userId no puede tener ya un ServiceProvider registrado
- **Respuesta:** El objeto ServiceProvider creado

PUT `/api/services/provider/kyc`

- **Quién puede llamarla:** El propio proveedor (rol PROVIDER)
- **Qué hace:** Actualiza campos del KYC. Cuando todos los campos obligatorios estén llenos, cambia kycStatus a SUBMITTED y status a PENDING_APPROVAL
- **Campos obligatorios para considerar KYC completo:**
 - idDocumentUrl, selfieWithIdUrl, professionalTitleUrl (al menos 1), licenseNumber, licenseIssuedBy
- **Efecto secundario:** Envía notificación a todos los ADMIN/SUPER_ADMIN de que hay un KYC pendiente

PUT `/api/services/provider/profile`

- **Quién puede llamarla:** El propio proveedor (status ACTIVE requerido)
- **Qué hace:** Actualiza datos de perfil (fotos, contacto, bio, redes sociales)
- **Restricción:** No puede actualizar campos de KYC después de aprobado (solo Admin puede)

GET `/api/services/provider/:providerId`

- **Quién puede llamarla:** Cualquier usuario autenticado
- **Qué devuelve:** Perfil público del proveedor (sin internalPrice, sin datos KYC privados)

GET `/api/services/provider/:providerId/admin`

- **Quién puede llamarla:** ADMIN, SUPER_ADMIN, ACCOUNTANT

- **Qué devuelve:** Perfil completo incluyendo documentos KYC, precios internos, convenio

POST /api/admin/providers/:providerId/approve

- **Quién puede llamarla:** ADMIN, SUPER_ADMIN
- **Qué hace:** Cambia kycStatus a APPROVED, status a ACTIVE, genera agreementNumber único, registra quién aprobó y cuándo
- **Efecto secundario:** Envía notificación de aprobación al proveedor (email + WhatsApp)

POST /api/admin/providers/:providerId/reject

- **Body:** `{ reason: string }`
- **Quién puede llamarla:** ADMIN, SUPER_ADMIN
- **Qué hace:** Cambia kycStatus a REJECTED, status a REJECTED, guarda razón del rechazo
- **Efecto secundario:** Notifica al proveedor con razón del rechazo

2.2 API de Listado de Servicios

POST /api/services/listings

- **Quién puede llamarla:** Proveedor ACTIVE
- **Qué hace:** Crea un nuevo servicio. Si es el primer servicio, pasa a revisión de admin (requiresApproval=true automáticamente en el primer lote)
- **Validaciones críticas:**
 - $memberPrice < publicPrice$ (el precio de miembro SIEMPRE debe ser menor al público)
 - $internalPrice < memberPrice$ (el precio interno SIEMPRE es el más bajo)
 - $companyCommission = memberPrice - internalPrice$ (calculado automáticamente por el servidor, no por el cliente)
 - ivaAmount se calcula en servidor: $internalPrice * (ivaPercentage / 100)$
- **NUNCA confíes en los cálculos del frontend para precios**

GET /api/services/listings?category=&city=&search=&page=

- **Quién puede llamarla:** Cualquier persona (incluso sin login)
- **Qué devuelve:**
 - Si no está autenticado: muestra publicPrice
 - Si está autenticado sin membresía: muestra publicPrice
 - Si está autenticado con membresía PREFERENTE o PIONERO: muestra memberPrice con badge "Precio Exclusivo"

- NUNCA devuelve internalPrice al frontend

PUT /api/services/listings/:listingId

- **Quién puede llamarla:** El proveedor dueño del servicio
- **Qué hace:** Guarda los cambios en el campo `pendingUpdate` (no sobrescribe el servicio activo). El Admin debe aprobar el cambio antes de que se publique.
- **Efecto secundario:** Notifica a Admin que hay un cambio pendiente

POST /api/admin/listings/:listingId/approve-update

- **Quién puede llamarla:** ADMIN
- **Qué hace:** Aplica el `pendingUpdate` al servicio activo, limpia pendingUpdate

2.3 API de Beneficiarios Familiares

POST /api/user/beneficiaries

- **Quién puede llamarla:** Usuario con membresía PREFERENTE o PIONERO
- **Límite:** Máximo 5 beneficiarios por cuenta. Si ya tiene 5, retornar error claro: "Has alcanzado el límite de 5 beneficiarios familiares."
- **Qué hace:** Crea FamilyBeneficiary, sube fotos a Supabase Storage
- **Validaciones:** idDocumentNumber no puede repetirse en la misma cuenta

GET /api/user/beneficiaries

- **Quién puede llamarla:** El propio usuario
- **Qué devuelve:** Lista de sus beneficiarios con estado y fotos

PUT /api/user/beneficiaries/:beneficiaryId

- **Qué hace:** Actualiza datos del beneficiario

DELETE /api/user/beneficiaries/:beneficiaryId

- **Qué hace:** Desactiva el beneficiario (isActive=false). No borra de la DB para mantener historial de citas anteriores.

2.4 API del Sistema de Citas

POST /api/appointments

- **Quién puede llamarla:** Usuario autenticado con membresía activa

- **Body:**

json

```
{
  "serviceId": "...",
  "beneficiaryId": "... | null",
  "requestedDate": "2026-05-15",
  "requestedTimeSlot": "10:00-11:00",
  "clientNotes": "...",
  "isEmergency": false
}
```

- **Proceso:**

1. Validar que el servicio existe y está ACTIVE
2. Validar que el cliente tiene membresía activa (o es cliente externo que paga precio público)
3. Guardar precios vigentes en el momento de la solicitud (snapshot de precios)
4. Crear Appointment con status PENDING_PROVIDER
5. Notificar al proveedor por todos sus canales registrados (email, WhatsApp, push)

- **Para emergencias (isEmergency=true):**

- Cambiar status directamente a EMERGENCY
- Enviar notificación URGENTE al proveedor (si el servicio tiene allowEmergency=true)
- Si el servicio no acepta emergencias, retornar error claro

GET /api/appointments/:appointmentId

- **Quién puede llamarla:** El cliente de la cita, el proveedor de la cita, o Admin
- **Validación:** No puede ver la cita alguien que no sea parte de ella (excepto Admin)

POST /api/appointments/:appointmentId/provider-respond

- **Quién puede llamarla:** El proveedor de esa cita
- **Body:**

json

```

{
  "proposedSlots": [
    {"date": "2026-05-15", "time": "09:00", "available": true},
    {"date": "2026-05-15", "time": "14:00", "available": true},
    {"date": "2026-05-16", "time": "10:00", "available": true}
  ],
  "providerNotes": "..."
}

```

- **Qué hace:** Actualiza status a PROVIDER_RESPONDED, guarda proposedSlots, notifica al cliente

POST /api/appointments/:appointmentId/client-confirm

- **Quién puede llamarla:** El cliente de esa cita
- **Body:** { "selectedSlot": {"date": "...", "time": "..."} }
- **Qué hace:**
 - Guarda confirmedDate
 - Si requiresPrePayment=false: status → CONFIRMED, notifica al proveedor
 - Si requiresPrePayment=true: status → PENDING_CLIENT (espera pago), redirige al flujo de pago
- **Regla importante:** Una vez que el status llega a CONFIRMED con prepago, NO SE PUEDE CANCELAR

POST /api/appointments/:appointmentId/pay

- **Quién puede llamarla:** El cliente (solo si status=PENDING_CLIENT y requiresPrePayment=true)
- **Body:** { "paymentMethod": "...", "paymentDetails": {...} }
- **Qué hace:**
 - Procesa el pago por el método elegido
 - Si pago exitoso: status → PAID, notifica al proveedor y cliente
 - Genera recibo de prepago con todos los detalles e IVA
 - Envía recibo por email + WhatsApp + Telegram al cliente
 - **IMPORTANTE:** En el recibo y en la notificación de confirmación, mostrar el siguiente aviso obligatorio:

" ⚠ AVISO IMPORTANTE: El pago de este servicio es definitivo. No se aceptan cancelaciones ni devoluciones una vez confirmado el pago. Eres responsable de presentarte a tu cita en la fecha y hora acordadas."

POST /api/appointments/:appointmentId/cancel

- **Quién puede llamarla:** Cliente o Proveedor
- **Reglas de cancelación:**
 - Cliente puede cancelar SOLO si status=PENDING_PROVIDER o PROVIDER_RESPONDED (antes de confirmar)
 - Si ya está CONFIRMED o PAID (prepago): el cliente NO puede cancelar desde la plataforma. Debe contactar a soporte.
 - Proveedor puede cancelar en cualquier status previo a COMPLETED, pero se registra en su historial (impacta su reputación si es frecuente)
- **Si el proveedor cancela:** Notificar al cliente inmediatamente, registrar en historial del proveedor

POST /api/appointments/:appointmentId/qr-scan

- **Quién puede llamarla:** El proveedor (desde su terminal/app móvil)
- **Body:** { "qrToken": "... " } — el token extraído del QR del cliente
- **Proceso:**
 1. Decodificar el QR y obtener el userId del titular
 2. Verificar que la cita corresponde a ese usuario o a uno de sus beneficiarios
 3. Mostrar al proveedor: foto del titular + fotos y datos del beneficiario si corresponde
 4. Cambiar status a IN_PROGRESS
 5. Registrar actualStartTime
 6. Notificar al cliente que el proveedor inició la atención

POST /api/appointments/:appointmentId/bipartite

- **Quién puede llamarla:** El proveedor (para crear/llevar) y el cliente (para aceptar)
- **Fase 1 - Proveedor llena:**
 - Body: datos del formulario bipartito
 - Crea BipartiteForm con status PROVIDER_FILLING → PENDING_CLIENT_ACCEPTANCE
 - Notifica al cliente para que acepte
- **Fase 2 - Cliente acepta/rechaza:**
 - Body: { "action": "ACCEPT" | "REJECT", "rejectionReason": "... " }
 - Si ACCEPT: formStatus → BOTH_SIGNED, status de cita → COMPLETED, dispara generación de factura, libera fondos del proveedor, registra comisión de empresa

- Si REJECT: formStatus → DISPUTED, el proveedor cobra precio público, notifica a Admin
- Registra IP, timestamp y "firma digital" de ambas partes

2.5 API de QR del Usuario

GET /api/user/qr-token

- **Quién puede llamarla:** El usuario autenticado
- **Qué hace:** Genera un token QR temporal (válido 24 horas) firmado con JWT

```
Payload del JWT:  
{  
  userId: "...",  
  memberType: "PREFERENTE" | "PIONERO",  
  membershipExpiresAt: "...",  
  isActive: true,  
  iat: ...,  
  exp: ... (24 horas)  
}
```

- **Qué devuelve:** La cadena del QR para generar el código visual en el frontend

POST /api/services/provider/scan-qr

- **Quién puede llamarla:** Proveedor ACTIVE
- **Body:** { "qrToken": "...", "appointmentId": "..." }
- **Proceso:**
 1. Verificar y decodificar el JWT
 2. Verificar que la membresía no ha expirado
 3. Buscar cita activa para ese usuario con ese proveedor
 4. Devolver: datos del titular + lista de beneficiarios + foto + estado de membresía
 5. Llamar internamente a **/api/appointments/:id/qr-scan**

2.6 API de Facturación

POST /api/invoices/service (llamada interna, no expuesta directamente)

- **Cuándo se llama:** Automáticamente cuando BipartiteForm llega a status BOTH_SIGNED
- **Qué hace:**

1. Calcula todos los montos (subtotal, IVA, total, comisión empresa, pago neto proveedor)
2. Genera número de factura secuencial: SC-SRV-{año}{mes}{secuencial 6 dígitos}
3. Genera PDF de factura con todos los datos (usa librería `@react-pdf/renderer` o `pdfkit`)
4. Sube PDF a Supabase Storage
5. Guarda URL en ServiceInvoice
6. Envía PDF por email al cliente Y al proveedor
7. Registra la comisión en el sistema contable interno (tabla interna, no visible en front)
8. Si el servicio fue prepago: marca los fondos como liberados al proveedor

`GET /api/invoices/:invoiceId`

- **Quién puede llamarla:** Cliente de la factura, proveedor de la factura, Admin
- **Qué devuelve:**
 - Para cliente/proveedor: factura pública (subtotal, IVA, total)
 - Para Admin/Accountant: factura completa (incluyendo comisión empresa y pago neto proveedor)

2.7 API de Reseñas

`POST /api/reviews/provider`

- **Quién puede llamarla:** Cliente (24 horas después de que status=COMPLETED)
- **Validaciones:**
 - La cita debe estar COMPLETED
 - El cliente no puede haber dado reseña ya para esa cita
 - rating debe ser 1-5
 - comment mínimo 20 caracteres
- **Qué hace:** Crea ProviderReview, actualiza promedio de calificación del proveedor

`POST /api/reviews/client`

- **Quién puede llamarla:** Proveedor (24 horas después de que status=COMPLETED)
- **Qué hace:** Crea ClientReview — solo visible para otros proveedores de la plataforma
- **IMPORTANTE:** El cliente NO puede ver las reseñas que le hacen los proveedores desde su dashboard. Solo Admin y otros proveedores las ven.

GET /api/reviews/provider/:providerId

- **Devuelve:** Todas las reseñas visibles del proveedor (promedio, distribución, comentarios)

2.8 API de Moderación y Sanciones

POST /api/admin/warnings/provider

- **Body:** { providerId, type, reason, details, expiresAt? }
- **Efecto:** Crea ProviderWarning, si type=SUSPENSION_TEMP actualiza provider.status, notifica al proveedor

POST /api/admin/warnings/client

- **Body:** { clientId, type, reason, details, expiresAt? }
- **Efecto:** Crea ClientWarning, si type=SUSPENSION_PERM bloquea al usuario

PARTE 3 — FLUJOS DE NEGOCIO COMPLETOS

INSTRUCCIÓN: Implementa estos flujos exactamente como se describen. Cada paso debe estar cubierto por código y por prueba automatizada.

FLUJO A — Onboarding de Nuevo Proveedor de Servicios

PASO 1: El usuario con rol PROVIDER accede a /dashboard/provider/onboarding

PASO 2: El sistema muestra el stepper: [Información Básica] → [KYC] → [Servicios] → [Revisión]

PASO 3: INFORMACIÓN BÁSICA (ServiceProvider - campos básicos)

- businessName, profession, professionCategory
- phone, email, whatsapp, address, city
- bio
- Guardar parcialmente → status queda PENDING_KYC

PASO 4: KYC PROFESIONAL

El formulario tiene secciones según professionCategory:

SI professionCategory = HEALTH:

- Solicitar: Número de registro SENESCYT, universidad, año de graduación
- Documentos: Título profesional, certificado SENESCYT, certificado ACCESS si aplica

SI professionCategory = LEGAL:

- Solicitar: Número de registro en el Foro de Abogados, título de abogado

- Documentos: Título de abogado, certificado del Foro

SI professionCategory = ARCHITECTURE o ENGINEERING:

- Solicitar: Número de registro en el CIAP (Colegio de Ingenieros/Arquitectos)
- Documentos: Título, registro colegial vigente

PARA TODOS:

- Foto de cédula frente y reverso
- Selfie sosteniendo la cédula
- Hasta 3 fotos personales
- Hasta 3 fotos de trabajo
- Hasta 3 fotos del negocio/local

Al completar y enviar → kycStatus = SUBMITTED, status = PENDING_APPROVAL

El sistema notifica a todos los ADMIN por email + push

PASO 5: REVISIÓN DE ADMIN (panel interno)

Admin ve lista de KYC pendientes

Admin puede:

- a) APROBAR → status = ACTIVE, se genera agreementNumber único
- b) RECHAZAR → status = REJECTED con razón
- c) SOLICITAR ACTUALIZACIÓN → kycStatus = REQUIRES_UPDATE, se notifica al proveedor

PASO 6: NOTIFICACIÓN AL PROVEEDOR

Si aprobado: "¡Felicidades! Tu perfil ha sido aprobado. Tu número de convenio es [X]. Ya puedes agregar tus servicios."

Si rechazado: "Tu solicitud fue rechazada por: [razón]. Puedes volver a aplicar corrigiendo los datos indicados."

PASO 7: AGREGAR SERVICIOS (solo si status=ACTIVE)

El proveedor crea sus servicios con los 3 niveles de precio

El sistema calcula automáticamente $companyCommission = memberPrice - internalPrice$

Los primeros servicios quedan con requiresApproval=true hasta que Admin los apruebe

FLUJO B — Registro de Beneficiarios Familiares

PASO 1: Usuario con membresía va a /dashboard/familia

PASO 2: El sistema muestra:

- Los beneficiarios ya registrados (hasta 5)
- Un botón "Agregar familiar" (deshabilitado si ya tiene 5)

PASO 3: Formulario de nuevo beneficiario:

- firstName, lastName
- relationship (qué parentesco tiene)
- dateOfBirth
- idDocumentNumber, idDocumentType
- Subir foto del familiar
- Subir imagen de cédula

PASO 4: El sistema valida:

- El usuario no tenga ya 5 beneficiarios activos
- El idDocumentNumber no esté ya registrado en esa cuenta
- Si pasa: crea FamilyBeneficiary con isActive=true

PASO 5: El dashboard muestra para cada familiar:

- Foto, nombre, relación, documento
- Estado (activo/inactivo)
- Historial de servicios recibidos con ese beneficiario

NOTA TÉCNICA DEL QR: El QR del titular es UNO SOLO. Cuando un proveedor lo escanea, el sistema muestra el titular Y la lista de sus beneficiarios activos. El proveedor selecciona si la atención es para el titular o para cuál de los familiares.

FLUJO C — Solicitud de Servicio (Flujo Normal, Sin Prepago)

PASO 1: Cliente navega el Marketplace de Servicios

URL: /servicios

Puede filtrar por: categoría, ciudad, nombre, calificación, precio

PASO 2: El cliente ve la tarjeta del servicio

- Nombre del servicio
- Nombre del proveedor y su foto
- Calificación promedio (estrellas)
- Modalidad (presencial/virtual/domicilio)
- Precio público: \$50.00
- [Si tiene membresía activa] Badge verde "Precio exclusivo SaidonClub: \$40.00"
- Duración estimada: 60 min
- Botón "Solicitar servicio"

PASO 3: El cliente hace clic en "Solicitar servicio"

Si NO está autenticado → redirige a login con retorno a la misma página

Si está autenticado → abre modal/página de solicitud

PASO 4: Formulario de solicitud

- Confirmar si es para el titular o para un beneficiario familiar
(si elige beneficiario, muestra lista de sus familiares registrados)
- Fecha preferida (date picker con días disponibles del proveedor)

- Franja horaria preferida (según schedule del proveedor)
- Notas adicionales (opcional, 500 chars máx)
- Checkbox obligatorio: "He leído y acepto los términos del servicio de SaidonClub"
- Botón "Enviar solicitud"

PASO 5: Sistema crea Appointment con status=PENDING_PROVIDER

- Guarda snapshot de precios vigentes
- Notifica al proveedor: email + WhatsApp + push con todos los datos de la cita solicitada

PASO 6: El proveedor responde (desde su panel /dashboard/provider/citas)

El proveedor ve la solicitud y puede:

- PROPONER HORARIOS: selecciona 2 o 3 slots disponibles de su calendario
 - status = PROVIDER_RESPONDED
 - el cliente recibe notificación con los slots propuestos
- RECHAZAR: con razón (ej: "No tengo disponibilidad esa semana")
 - status = CANCELLED_PROVIDER
 - el cliente recibe notificación de rechazo con razón

PASO 7: El cliente elige horario

El cliente ve los slots propuestos y hace clic en el que prefiere

- Confirma la cita
- status = CONFIRMED
- El proveedor recibe notificación de confirmación
- Ambos reciben un resumen completo de la cita por email + WhatsApp

PASO 8: Recordatorios automáticos

- 24 horas antes: notificación a cliente y proveedor
- 1 hora antes: notificación a cliente y proveedor

PASO 9: El día de la cita

El proveedor abre su app/terminal y escanea el QR del cliente

El sistema muestra:

- Foto del titular
- Si la cita es para un beneficiario: foto y datos del beneficiario
- Datos de membresía
- Cita confirmada para hoy
- status = IN_PROGRESS

PASO 10: Al finalizar el servicio – Formulario Bipartito

El proveedor en su terminal llena:

- Descripción del servicio realizado
- ¿Hubo servicios adicionales? (si/no, con detalle)
- Monto total cobrado (el sistema calcula con IVA)
- Forma de pago (efectivo/tarjeta al proveedor / puntos SaidonClub)

El proveedor envía el formulario

→ El cliente recibe notificación: "Tu proveedor ha registrado el servicio. Por favor confirma."

El cliente en su teléfono ve el resumen y presiona "Confirmar y Aceptar"

→ formStatus = BOTH_SIGNED

→ status = COMPLETED

→ Se genera factura automáticamente

→ Se registra comisión de empresa (privado)

→ El sistema libera el pago neto al proveedor en su wallet

PASO 11: Reseñas (opcional, aparece 24h después)

- El cliente recibe invitación: "¿Cómo fue tu experiencia con [Proveedor]? Deja tu reseña"

- El proveedor recibe invitación: "¿Cómo fue tu cliente [Nombre]? Deja tu evaluación interna"

FLUJO D — Solicitud de Servicio (CON PREPAGO en plataforma)

Igual que Flujo C hasta PASO 7 (confirmación de horario), luego:

PASO 7b: En lugar de confirmar directamente, el sistema muestra:

"Este servicio requiere pago anticipado"

Resumen de lo que se va a pagar:

	Consulta Médica General	
	Dr. Juan Pérez	
	Fecha: Miércoles 20 de mayo, 10:00 AM	
	Precio exclusivo SaidonClub:	\$40.00
	IVA (15%):	\$6.00
	TOTAL A PAGAR:	\$46.00

 **AVISO IMPORTANTE — LEE ANTES DE PAGAR:**

"Una vez realizado el pago, la cita queda confirmada de forma definitiva.

No se aceptan cancelaciones ni devoluciones bajo ninguna circunstancia.

Eres responsable de presentarte puntualmente a tu cita.

Si no te presentas, perderás el monto pagado.

Si tienes alguna emergencia, contacta inmediatamente a soporte."

Checkbox obligatorio: "Entiendo y acepto que el pago es definitivo"

→ Métodos de pago (los 9 disponibles en la plataforma)

→ Botón "Confirmar y Pagar \$46.00"

PASO 7c: Pago procesado exitosamente

El sistema:

1. Confirma el pago
2. Cambia status = PAID
3. Genera recibo de prepago (PDF)
4. Envía recibo por email + WhatsApp + Telegram al cliente
5. Notifica al proveedor que el pago está confirmado
6. El dinero queda en "fondos retenidos" hasta que se complete el servicio

PASO 7d: El recibo de prepago incluye:

- Número de reserva: SC-APT-0000001
- Datos del cliente y proveedor
- Servicio reservado
- Fecha y hora confirmada
- Monto pagado con desglose de IVA
- Código QR de la reserva
- Condiciones de no-cancelación (en negrita)
- Instrucciones para el día de la cita

Continúa igual que Flujo C desde PASO 8 en adelante.

FLUJO E — Solicitud de Emergencia

PASO 1: Cliente en la página del servicio hace clic en "¡Emergencia!"
(este botón solo aparece si el servicio tiene allowEmergency=true)

PASO 2: Modal de emergencia:

"¿Confirmas que esto es una emergencia médica/legal/etc que no puede esperar?"

Campo: "Describe brevemente la emergencia" (obligatorio)

Botón: "Sí, es una emergencia"

PASO 3: Sistema crea Appointment con:

- isEmergency = true
- status = EMERGENCY
- Sin fecha/hora (es inmediata)

PASO 4: Notificación URGENTE al proveedor:

"🚨 EMERGENCIA: Un cliente necesita atención urgente ahora mismo.

Cliente: [Nombre]

Motivo: [descripción]

Teléfono: [teléfono del cliente]

Por favor responde lo antes posible."

Se envía por: push (con sonido de emergencia), WhatsApp, SMS, email

PASO 5: El proveedor responde ACEPTAR o NO PUEDE ATENDER

Si acepta: coordina directamente con el cliente (por teléfono/WhatsApp)

Si no puede: sistema busca automáticamente otro proveedor del mismo tipo

PASO 6: Después de atender, el proveedor registra el servicio en la plataforma
(puede hacerlo hasta 24 horas después de la atención)

Sigue el mismo flujo del formulario bipartito

El cliente tiene que confirmar desde su app (también hasta 24h después)

IMPORTANTE: Las emergencias NO requieren prepago.

El pago se registra como parte del formulario bipartito (postpago).

PARTE 4 — LÓGICA FINANCIERA INTERNA (CRÍTICA)

INSTRUCCIÓN: Esta sección describe cómo fluye el dinero internamente. Es información contable que NUNCA debe mostrarse al cliente. Solo Admin, Super Admin y Accountant pueden verla.

4.1 Ejemplo de Cálculo con el Doctor Oculista

Configuración del servicio:

```
publicPrice      = $50.00  (lo que ve el público)
memberPrice     = $40.00  (lo que ve el miembro SaidonClub)
internalPrice    = $30.00  (precio del convenio privado)
ivaPercentage    = 15%
ivaIncluded      = false   (el IVA se suma por encima)
```

Cuando un miembro SaidonClub solicita el servicio:

```
baseCharge       = memberPrice = $40.00
ivaAmount        = $40.00 * 0.15 = $6.00
totalCharged     = $46.00  (lo paga el cliente)
```

Distribución interna:

```
companyCommission = memberPrice - internalPrice = $40.00 - $30.00 = $10.00
providerNetAmount = internalPrice = $30.00
ivaToReport       = $6.00 (obligación tributaria del proveedor)
```

La empresa SaidonClub retiene:

- Su comisión de \$10.00
- Gestiona el IVA (\$6.00 para declaración)

El proveedor recibe en su wallet de la plataforma:

- \$30.00 (precio del convenio, sin IVA ya que el IVA lo declara el proveedor)

Registro contable (tabla interna):

```
{
  appointmentId: "...",
  agreementNumber: "Convenio Nro 362535",
  providerCode: "SC-PROV-8y489807sc",
  serviceDate: "2026-05-20",
  clientChargedTotal: 46.00,
  ivaAmount: 6.00,
  companyCommission: 10.00,
  providerNetPayment: 30.00,
  currency: "USD"
}
```

4.2 Regla del Precio Público

Si el cliente NO tiene membresía activa → se aplica publicPrice

Si el cliente rechaza el formulario bipartito → se aplica publicPrice

(En este caso, el proveedor cobra al cliente su precio normal de calle,
el sistema registra la transacción pero no procesa comisión para la empresa)

El sistema NUNCA cobra menos que publicPrice si el cliente no tiene membresía.

4.3 Wallet del Proveedor

Cuando un servicio es COMPLETADO (BipartiteForm = BOTH_SIGNED):

1. Se crea una transacción en el Wallet del proveedor:

type: SERVICE_PAYMENT

amount: providerNetAmount

status: AVAILABLE (si fue postpago)

status: RELEASED (si fue prepago, estaba retenido)

2. El proveedor puede retirar esos fondos mediante el sistema de retiros
ya existente en la plataforma.

3. La comisión de la empresa queda registrada en una tabla separada
para contabilidad (FundsReserve existente, agregar tipo SERVICE_COMMISSION)

PARTE 5 — FRONTEND / PÁGINAS Y COMPONENTES

INSTRUCCIÓN: Crea estas páginas dentro de `(apps/web/app/)`. Usa el sistema de componentes existente. Sigue el estándar visual "Obsidian & Orange" del Documento Maestro: fondos oscuros, degradados suaves, glassmorphism, tipografía fluida con clamp().

5.1 Marketplace de Servicios (`/servicios`)

Página principal `/servicios`

Elementos:

- Hero con buscador prominente
- Filtros: categoría (chips), ciudad (select), modalidad (chips), precio (slider)
- Grid de tarjetas de servicios
- Mapa opcional con ubicaciones de proveedores (Google Maps embed)
- Paginación

Tarjeta de servicio:

- Foto del proveedor (circular)
- Badge de categoría
- Nombre del servicio
- Nombre del proveedor
- Calificación (estrellas + número de reseñas)
- Ciudad y modalidad
- Precios (público siempre visible, precio miembro solo si está autenticado con membresía)
- Duración estimada
- Badge "Acepta emergencias" si aplica
- Botón "Ver detalles" y botón rápido "Solicitar"

Página de detalle de servicio `/servicios/[serviceId]`

Secciones:

1. Header: foto del proveedor + nombre + profesión + calificación + badges
2. Galería de fotos (fotos del proveedor, negocio, trabajo)
3. Descripción del servicio
4. Precios (con tabla de 3 columnas si el usuario es Admin/Accountant)
5. Información de contacto y ubicación (con mapa)
6. Horarios de atención
7. Formulario de solicitud de cita (o botón que abre el modal)
8. Reseñas de otros clientes (con paginación)
9. Otros servicios del mismo proveedor

Perfil público del proveedor `/proveedores/[providerId]`

Secciones:

1. Encabezado: foto + nombre + profesión + calificación global + convenio (NO mostrar número, solo "Proveedor Verificado SaidonClub")
2. Galería completa de fotos
3. Bio y credenciales (sin mostrar documentos privados, solo "Verificado por

SaidonClub")

- 4. Información de contacto y redes sociales
- 5. Lista de todos sus servicios activos
- 6. Reseñas

5.2 Dashboard del Proveedor

/dashboard/provider — Panel principal del proveedor

Bloques:

- Resumen financiero: ingresos del mes, comisiones ganadas, pendientes de pago
- Citas de hoy (fecha actual)
- Solicitudes pendientes de respuesta (alerta si hay más de 0)
- Calificación promedio + número de reseñas
- Estado del perfil (si KYC pendiente, mostrar alerta con pasos a seguir)

/dashboard/provider/citas — Gestión de citas

Tabs:

- Pendientes (PENDING_PROVIDER): mostrar con alarma si llevan más de 2 horas sin respuesta
 - Confirmadas: próximas citas
 - En curso (IN_PROGRESS)
 - Completadas
 - Canceladas/Rechazadas
 - Emergencias

Para cada cita pendiente, el proveedor puede:

- Proponer horarios (selector de fechas con su calendario de disponibilidad)
- Rechazar con razón

Para citas confirmadas:

- Ver detalles del cliente
- Botón "Iniciar atención" (que activa el escáner QR)

Para citas en curso:

- Botón "Llenar formulario de servicio" (bipartito)

/dashboard/provider/servicios — Mis servicios

- Lista de servicios publicados con estado (activo/inactivo/pendiente aprobación)
- Botón "Agregar nuevo servicio"
- Para cada servicio: botón editar (guarda como pendingUpdate, espera aprobación)

- Solicitar actualización de lista de servicios (para cuando hay cambios de precio o nuevos servicios)

/dashboard/provider/perfil — Mi perfil y KYC

- Stepper de KYC con estado actual
- Formulario de actualización de fotos y datos de contacto
- Documentos subidos (solo el proveedor los ve, admin también)
- Estado del convenio

/dashboard/provider/scanner — Escáner QR

- Interfaz de cámara para escanear QR del cliente
- Al escanear: muestra foto + nombre + membresía + beneficiarios
- Selector: "¿Esta atención es para...?" [El titular / Familiar 1 / Familiar 2 / ...]
- Botón "Confirmar inicio de atención"

5.3 Dashboard del Cliente (nuevos módulos)

/dashboard/familia — Mis beneficiarios familiares

- Header: "Mis beneficiarios (X/5)"
- Lista de beneficiarios con foto, nombre, relación, estado
- Botón "Agregar familiar" (deshabilitado con tooltip si ya tiene 5)
- Para cada familiar: botón editar, botón desactivar
- Historial de citas de cada familiar

/dashboard/mi-qr — Mi código QR

- Código QR grande y visible
- Nombre del titular
- Foto del titular
- Tipo de membresía
- Estado (activo/vencido)
- Botón "Descargar QR"
- Instrucciones: "Muestra este código al proveedor de servicios cuando vayas a tu cita"
- Botón "Renovar membresía" (si está vencida)

/dashboard/citas — Mis citas

Tabs: Próximas / Pasadas / Canceladas

Para citas próximas:

- Datos del servicio y proveedor
- Fecha y hora
- Estado con color
- Si está PENDING_PROVIDER: "Esperando respuesta del proveedor" + opción de cancelar
- Si está PROVIDER_RESPONDED: mostrar los horarios propuestos con botones para elegir
- Si está CONFIRMED: recordatorio de la cita
- Si está PAID (prepago): confirmación de pago + instrucciones
- Si está IN_PROGRESS: "Tu cita está en curso"
- Botón "Ver detalles completos"

Para citas con formulario bipartito pendiente:

- Alerta destacada: "Tu proveedor registró el servicio. Confírmalo aquí."
- Resumen del formulario bipartito
- Botón "Confirmar que recibí el servicio" y "Hay un problema (disputar)"

5.4 Panel de Administración de Servicios

`/admin/servicios/proveedores` — Lista de proveedores

- Tabla con filtros: por estado (KYC pendiente, activo, suspendido)
- Para cada proveedor: foto, nombre, profesión, estado, fecha de solicitud, acciones
- Botón "Revisar KYC" → lleva a página de revisión

`/admin/servicios/kyc/[providerId]` — Revisión de KYC

- Vista completa de todos los documentos subidos
- Visor de imagen para cada documento
- Verificación campo por campo con checkboxes
- Campo de notas del revisor
- Botones: "Aprobar", "Rechazar", "Solicitar correcciones"
- Historial de revisiones anteriores

`/admin/servicios/citas` — Vista global de citas

- Tabla con todas las citas del sistema
- Filtros: por estado, por proveedor, por fecha, por emergencia
- Vista de métricas: citas por día, tasa de completado, tasa de cancelación

`/admin/contabilidad/servicios` — Contabilidad interna de servicios

- Solo visible para ADMIN, SUPER_ADMIN, ACCOUNTANT
- Tabla de todas las facturas con columnas: subtotal, IVA, total cliente, comisión empresa, pago neto proveedor
- Filtros por proveedor, fecha, estado
- Exportar a CSV/Excel
- Totales acumulados

`/admin/moderacion` — Sistema de moderación

- Lista de reseñas reportadas
- Lista de citas en disputa
- Historial de amonestaciones y suspensiones
- Formulario para emitir nueva amonestación

PARTE 6 — DOCUMENTOS LEGALES DINÁMICOS

INSTRUCCIÓN: Estos documentos deben estar en la base de datos (tabla LegalDocument) para que puedan actualizarse sin desplegar código. Crear un CRUD para que el Super Admin los modifique. Deben mostrarse con fecha de última actualización.

6.1 Documentos Requeridos

Crea el modelo:

```
prisma

model LegalDocument {
  id          String    @id @default(cuid())
  slug        String    @unique // "terms-of-service", "provider-agreement", etc.
  title       String
  content      String    @db.Text // Contenido en Markdown
  version     String    // "1.0", "1.1", etc.
  isActive    Boolean   @default(true)
  updatedAt   DateTime  @updatedAt
  updatedBy   String?
  @@map("legal_documents")
}
```

Documentos a crear con contenido inicial:

DOCUMENTO 1: Términos y Condiciones Generales ((terms-of-service))

Incluir:

- Descripción de SaidonClub
- Definición de membresías y beneficios
- Reglas de uso de la plataforma
- Política de cancelaciones (ninguna en prepago)
- Responsabilidad limitada de SaidonClub como intermediario
- Jurisdicción: República del Ecuador

DOCUMENTO 2: Reglamento de Proveedores de Servicios ((provider-regulations))

Incluir:

- Requisitos para ser proveedor
- Proceso de verificación KYC
- Obligaciones del proveedor:
 - Mantener datos actualizados
 - Emitir factura por cada servicio
 - Cumplir con los precios declarados en la plataforma
 - Atender al cliente con membresía al precio de convenio
 - Reportar el servicio en la plataforma dentro de las 24 horas
- Sistema de calificaciones y consecuencias
- Sistema de amonestaciones y suspensiones
- Causales de suspensión permanente

DOCUMENTO 3: Reglamento de Clientes ((client-regulations))

Incluir:

- Derechos del cliente con membresía
- Obligaciones del cliente:
 - Presentarse puntualmente a las citas
 - Comportarse de manera respetuosa
 - No solicitar servicios con intención de cancelar
- Sistema de reseñas (que los proveedores los pueden calificar internamente)
- Consecuencias de mal comportamiento
- Política de no reembolso en servicios prepagados

DOCUMENTO 4: Convenio Marco de Proveedor ((provider-agreement))

(Este se genera dinámicamente con los datos del proveedor al momento de aprobación)

Incluir:

- Número de convenio único
- Datos del proveedor
- Tabla de servicios registrados con precios internos
- Comisiones pactadas
- Vigencia del convenio
- Firma digital de aceptación

DOCUMENTO 5: Aviso de Privacidad ((privacy-policy))

Incluir:

- Datos que se recopilan
- Uso de los datos
- Compartición con terceros (solo proveedores autorizados)
- Derechos del usuario (acceso, rectificación, cancelación)

PARTE 7 — SISTEMA DE NOTIFICACIONES MULTI-CANAL

INSTRUCCIÓN: Implementa un servicio centralizado de notificaciones. Cada notificación debe intentar enviarse por todos los canales configurados del usuario. Si un canal falla, registra el error pero continúa con los demás.

7.1 Servicio de Notificaciones ((packages/notifications/))

typescript

```
// packages/notifications/src/index.ts

interface NotificationPayload {
  userId: string;
  type: NotificationType;
  title: string;
  body: string;
  data?: Record<string, unknown>;
  urgency?: 'normal' | 'high' | 'emergency';
}

class NotificationService {
  async send(payload: NotificationPayload): Promise<void> {
    // 1. Obtener preferencias de notificación del usuario
    // 2. Enviar por email (Resend o SendGrid)
    // 3. Enviar por WhatsApp (WhatsApp Business API o Twilio)
    // 4. Enviar por Telegram (Telegram Bot API si el usuario tiene su chatId)
    // 5. Enviar push notification (si tiene token FCM/APNS)
    // 6. Guardar en SystemNotification en la DB
    // 7. Registrar éxito/fallo de cada canal
  }
}
```

7.2 Templates de Mensajes

Para cada tipo de notificación, define un template claro:

APPOINTMENT_REQUESTED (para el proveedor):

Asunto: "Nueva solicitud de cita - [Nombre del cliente]"

Cuerpo:

"Hola [Nombre del proveedor],

Tienes una nueva solicitud de cita:

Cliente: [Nombre]

Servicio: [Nombre del servicio]

Fecha preferida: [Fecha]

Horario preferido: [Horario]

Notas: [Notas del cliente]

Por favor responde en las próximas 2 horas para confirmar disponibilidad.

Accede a tu panel para responder: [link]"

APPOINTMENT_CONFIRMED (para el cliente):

Asunto: "✅ Cita confirmada - [Nombre del servicio]"

Cuerpo:

"¡Tu cita ha sido confirmada!



Fecha: [Fecha]



Hora: [Hora]



Proveedor: [Nombre]



Lugar: [Dirección o 'Virtual']"

Recuerda:

- Lleva tu código QR de SaidonClub (disponible en tu dashboard)
- Preséntate 5 minutos antes
- Si tienes alguna emergencia, contacta soporte al [número]"

BIPARTITE_PENDING (para el cliente):

Asunto: "Confirma tu servicio recibido - Acción requerida"

Cuerpo:

"Tu proveedor [Nombre] ha registrado el servicio brindado.

Por favor revisa y confirma que recibiste el servicio:

Servicio: [Descripción]

Monto: \$[Total]

Tienes 24 horas para confirmar o disputar.

Confirmar: [link] | Disputar: [link]"

PARTE 8 — PRUEBAS AUTOMATIZADAS REQUERIDAS

INSTRUCCIÓN: Crea pruebas para cada flujo crítico. Usa Jest o Vitest. Las pruebas deben correr con `npm test` desde la raíz del monorepo. NO hagas commit de código sin pruebas que pasen.

8.1 Pruebas de Modelo de Datos

```
test: "ServiceProvider no puede tener memberPrice mayor o igual a publicPrice"
test: "ServiceProvider no puede tener internalPrice mayor o igual a memberPrice"
test: "companyCommission se calcula correctamente como memberPrice -
internalPrice"
test: "Un usuario no puede tener más de 5 FamilyBeneficiary activos"
test: "FamilyBeneficiary con mismo idDocumentNumber no puede registrarse en la
misma cuenta"
test: "Un usuario sin membresía activa no puede ver memberPrice en el marketplace"
```

8.2 Pruebas de Flujo de Citas

```
test: "Al crear appointment, los precios se guardan como snapshot y no cambian si
el proveedor actualiza precios"
test: "Un appointment en status PAID no puede ser cancelado por el cliente"
test: "Al completar BipartiteForm con BOTH_SIGNED, se genera ServiceInvoice
automáticamente"
test: "Al completar BipartiteForm, el providerNetAmount se acredita al wallet del
proveedor"
test: "Si el cliente rechaza el bipartito, se aplica publicPrice y no se genera
comisión"
test: "El QR token expira después de 24 horas"
test: "El proveedor solo puede escanear el QR de clientes que tienen cita
confirmada con él ese día"
```

8.3 Pruebas de Seguridad

```
test: "El internalPrice NUNCA aparece en la respuesta de API de listado público"
test: "El internalPrice NUNCA aparece en la respuesta de API para el cliente"
test: "Las ClientReview NO aparecen en el dashboard del cliente"
test: "Un proveedor no puede ver las citas de otro proveedor"
test: "Un cliente no puede ver las citas de otro cliente"
test: "El formulario bipartito solo puede ser aceptado por el cliente
correspondiente"
```

8.4 Pruebas de Cálculos Financieros

```
test: "El IVA se calcula correctamente al 15% sobre el memberPrice"
test: "El providerNetAmount = internalPrice (sin IVA)"
test: "La factura generada tiene los montos correctos"
test: "Si el servicio tiene ivaIncluded=true, el IVA se extrae del precio, no se suma"
```

PARTE 9 — GUÍA DE IMPLEMENTACIÓN PASO A PASO

INSTRUCCIÓN: Implementa en este orden. Completa y prueba cada fase antes de pasar a la siguiente. Haz un commit por cada fase completada.

Fase 1: Modelos de Base de Datos (estimado: 1 día)

1. Agrega todos los modelos de la PARTE 1 al schema.prisma
2. Ejecuta `prisma migrate dev --name add-services-marketplace`
3. Verifica que todas las relaciones estén correctas con `prisma validate`
4. Crea seeds de datos de prueba para: 2 proveedores con KYC completo, 3 servicios cada uno, 2 usuarios con membresía y beneficiarios familiares

Fase 2: APIs de Proveedor y KYC (estimado: 2 días)

1. Implementa todas las APIs de la sección 2.1
2. Implementa el flujo de subida de archivos a Supabase Storage
3. Implementa el panel de revisión de KYC para Admin
4. Prueba el flujo completo: registro → KYC → aprobación → notificación

Fase 3: APIs de Servicios y Calendario (estimado: 1 día)

1. Implementa APIs de ServiceListing (sección 2.2)
2. Implementa ProviderSchedule y ProviderBlockedDate
3. Implementa la lógica de slots disponibles
4. Prueba que los 3 niveles de precio funcionan correctamente y que el internalPrice nunca se filtra al cliente

Fase 4: APIs de Beneficiarios y QR (estimado: 1 día)

1. Implementa APIs de FamilyBeneficiary (sección 2.3)
2. Implementa generación de QR con JWT (sección 2.5)

3. Implementa el endpoint de escaneo de QR
4. Prueba que el QR muestra los datos correctos y que expira

Fase 5: Flujo Completo de Citas (estimado: 3 días)

1. Implementa todas las APIs de la sección 2.4 en orden
2. Prueba el Flujo C completo (sin prepago)
3. Prueba el Flujo D completo (con prepago)
4. Prueba el Flujo E (emergencia)
5. Verifica que las notificaciones se envían en cada paso

Fase 6: Formulario Bipartito y Facturación (estimado: 2 días)

1. Implementa el BipartiteForm (ambas fases)
2. Implementa la generación de PDF de factura
3. Implementa el crédito al wallet del proveedor
4. Prueba el flujo completo desde QR hasta factura

Fase 7: Reseñas y Moderación (estimado: 1 día)

1. Implementa APIs de reseñas (sección 2.7)
2. Implementa APIs de moderación (sección 2.8)
3. Verifica que las ClientReview son invisibles para los clientes

Fase 8: Frontend — Marketplace de Servicios (estimado: 3 días)

1. Página `/servicios` con filtros y grid
2. Página `/servicios/[serviceId]`
3. Modal de solicitud de cita
4. Flujo de confirmación y pago

Fase 9: Frontend — Dashboard del Proveedor (estimado: 2 días)

1. Panel principal con métricas
2. Gestión de citas
3. Escáner QR
4. Formulario bipartito
5. Mis servicios

Fase 10: Frontend — Dashboard del Cliente (nuevos módulos) (estimado: 2 días)

1. `/dashboard/familia`
2. `/dashboard/mi-qr`
3. `/dashboard/citas` (actualizado)
4. Confirmación de formulario bipartito

Fase 11: Panel de Administración (estimado: 2 días)

1. Revisión de KYC
2. Aprobación de servicios
3. Vista global de citas
4. Contabilidad interna
5. Moderación

Fase 12: Sistema de Notificaciones (estimado: 1 día)

1. Configura el servicio de email (Resend/SendGrid)
2. Configura WhatsApp Business API
3. Crea todos los templates
4. Conecta notificaciones a todos los flujos

Fase 13: Documentos Legales (estimado: 1 día)

1. Crea el modelo LegalDocument
2. Crea el CRUD de admin
3. Escribe el contenido inicial de los 5 documentos
4. Conecta los checkboxes de aceptación al registro y a los flujos de cita

Fase 14: Pruebas Completas y QA (estimado: 2 días)

1. Ejecuta todas las pruebas de la PARTE 8
2. Prueba de punta a punta: nuevo proveedor → KYC → aprobación → publicar servicio → cliente solicita → confirma → paga → QR → bipartito → factura → reseña
3. Prueba todos los flujos de error (qué pasa si el cliente rechaza el bipartito, si el proveedor cancela, si el QR expira, etc.)

1. **El internalPrice NUNCA llega al frontend del cliente o del proveedor público.** Solo admin/accountant lo ven.
 2. **Los cálculos de precios SIEMPRE se hacen en el servidor.** El frontend nunca debe calcular precios finales.
 3. **Las transacciones financieras son ACID.** Usa transacciones de Prisma para: acreditar wallet + registrar comisión + actualizar cita (todo en un solo bloque atómico).
 4. **Las ClientReview son INVISIBLES para el cliente.** Solo las ven otros proveedores y el Admin.
 5. **El QR es del titular y cubre a sus beneficiarios.** No hay un QR por beneficiario. Es un QR por cuenta.
 6. **Un servicio prepago confirmado NO SE PUEDE CANCELAR desde la plataforma.** El cliente debe ir a soporte.
 7. **El proveedor SIEMPRE debe dar factura.** El sistema lo recuerda y registra la obligación.
 8. **Todos los cambios de precios de servicios requieren aprobación del Admin.** El proveedor no puede cambiar precios unilateralmente y en tiempo real.
 9. **El IVA se calcula siempre al 15% (Ecuador).** Si esto cambia en el futuro, debe estar en una variable de configuración, no hardcodeado.
 10. **Todo evento relevante queda registrado.** Si no está en la DB, no ocurrió.
-

PARTE 11 — VARIABLES DE ENTORNO ADICIONALES REQUERIDAS

env

```
# WhatsApp Business API
WHATSAPP_API_TOKEN=
WHATSAPP_PHONE_NUMBER_ID=

# Telegram Bot
TELEGRAM_BOT_TOKEN=

# Email (Resend o SendGrid)
RESEND_API_KEY=
EMAIL_FROM_ADDRESS=noreply@saidonclub.com

# Supabase Storage (para archivos de KYC)
SUPABASE_STORAGE_BUCKET_PROVIDERS=saidonclub-providers
SUPABASE_STORAGE_BUCKET_USERS=saidonclub-users

# JWT para QR
QR_JWT_SECRET=
QR_JWT_EXPIRY=86400 # 24 horas en segundos

# IVA configuración
IVA_PERCENTAGE=15
IVA_INCLUDED_BY_DEFAULT=false

# Límites
MAX_FAMILY_BENEFICIARIES=5
MAX_PERSONAL_PHOTOS=3
MAX_WORK_PHOTOS=3
MAX_BUSINESS_PHOTOS=3
APPOINTMENT_QR_EXPIRY_HOURS=24
REVIEW_WINDOW_HOURS=72 # horas después del servicio para dejar reseña
BIPARTITE_ACCEPT_WINDOW_HOURS=24 # horas para que el cliente acepte el bipartito
```

Documento generado: 2026-05-01 Versión: 2.0 Contexto base: Documento Maestro SaidonClub v6.0 (2359 líneas) + Análisis Comparativo de Implementación Este documento es instrucción directa para el agente OpenCode